

```
--url http://18.212.88.185:8001/consumers

#Create a key for the consumer to access the service using
apikey
curl --header "Content-Type: application/json" \
  --request POST \
  --data '{"key" : "rkkrishnaa123"}' \
  --url http://18.212.88.185:8001/consumers/rkkrishnaa/
key-auth

#Ensure the consumer can access the resources with proper
apikey
curl --header "Content-Type: application/json" \
  --header "Host: microservices.io" \
  --header "apikey: rkkrishnaa123" \
  --request GET \
  --url http://18.212.88.185:8000/
```

Similarly, one can perform multiple actions to add, update, list and delete services, routes and consumers. You can perform active and passive health checks against your endpoints. One can do load balancing and proxy the requests to different upstream servers.

Scaling a Kong node is very easy — just add new servers behind the load balancer. Each node in Kong is stateless and is independent of others while handling requests from microservices clients. Ensure the availability of a data store. It is the backbone of the API gateway. For production deployments, it is better to use Cassandra as a data store. The Cassandra database can be scaled very easily because of its distributed architecture.

## The AWS API gateway

The Amazon API gateway is a fully managed service that makes it easy for developers to create, publish, maintain,

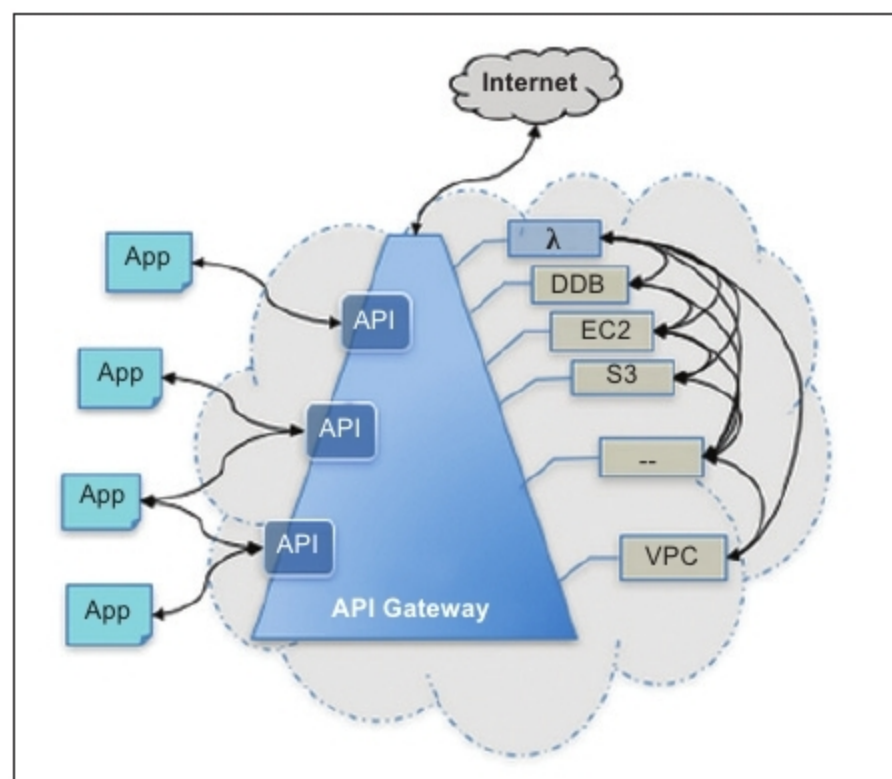


Figure 5: AWS API gateway architecture

monitor and secure APIs at any scale. With a few clicks in the AWS Management Console, one can create an API that acts as a ‘front door’ for applications to access data, business logic, or functionality from back-end services, such as workloads running on Amazon Elastic Compute Cloud (Amazon EC2), code running on AWS Lambda, or any Web application.

The Amazon API gateway handles all the tasks involved in accepting and processing up to hundreds of thousands of concurrent API calls, including traffic management, authorisation and access control, monitoring and API version management.

**Benefits of the AWS API gateway:** This gateway has quite a few benefits, as listed below.

*Low cost and efficient:* The Amazon API gateway has no minimum fees or startup costs. You pay only for the API calls received and the amount of data transferred out.

*Streamlines API development:* One can simultaneously run multiple versions of the same API, allowing quick iteration, testing and releases of new versions.

*Performance at any scale:* One can integrate the API with cloudfront edge locations and cache the output of API calls to avoid calling backend systems unnecessarily.

*Easily monitors API activity:* After the API is deployed, the Amazon API gateway provides a dashboard for visually monitoring calls to services using Amazon CloudWatch; so one can see the performance metrics and information on API calls, data latency and error rates.

*Runs APIs without servers:* Amazon API gateway tightly integrates with AWS Lambda to allow the creation of completely serverless APIs. With this gateway, one can create REST APIs that mobile and Web applications can use to call publicly available AWS services through the code running in AWS Lambda.

*Flexible security controls:* The API gateway provides tools to authorise access to your APIs and control service operations. AWS administration and security tools, such as AWS IAM and Amazon Cognito, can be used to authorise access to your APIs. It supports OAuth tokens as well.

**How to manage APIs in the AWS API gateway:** We can use the AWS API gateway console, AWS SDK or CLI to create, configure, update, deploy and delete an API. These three tools call the API gateway REST API behind the scenes to make things happen. Instead, we can call the API gateway REST API directly to create, configure, update, and deploy an API in API Gateway. While creating a new API, different options are available such as Clone from existing APIs and Import from Swagger. You can create a custom API which should point to the Lambda function or any other external endpoints, and can proxy API requests in the API gateway.

It is possible to create a serverless environment with the API gateway and Lambda. You can use DynamoDB to act as a data store for persisting application transactions. Multiple versions of the same API can be managed at different deployment stages, and each version of the API can be pointed to a different back-end.